

ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ И КОМПЬЮТЕРНОЕ ЗРЕНИЕ

РУТОРСН

Киселев А.В.



ДВА ФРЕЙМВОРКА



TensorFlow

PYTORCH VS TENSORFLOW

	PyTorch	Tensorflow
Программирование	Классы и объекты	Keras
Компьютерное зрение	torchvision, detectron, MMDetection	TF Hub, Google
Отладка	pdb	tf.print



PYTORCH VS TENSORFLOW

PyTorch vs Tensorflow

Производительность(CPU,
GPU)

Одинаковая

Мобильные устройства

TFLite, PyTorch
Mobile

Инференс

ONNX

Браузер

Tensorflow.js



PYTORCH VS TENSORFLOW

Keras

```
1 model = tf.keras.applications.ResNet50(weights='imagenet')
2 model.compile(optimizer='adam', loss='categorical_crossentropy')
3 model.fit(train_dataset, epochs=10)
```

Torch

```
1 model = torchvision.models.resnet50(weights='IMAGENET1K_V1')
2 optimizer = torch.optim.Adam(model.parameters())
3 for epoch in range(10):
4     for images, labels in dataloader:
5         outputs = model(images)
6         loss = criterion(outputs, labels)
7         loss.backward()
8         optimizer.step()
```

PYTORCH



TENSORS

Тензор - многомерный массив(аналог ndarray),
который может быть размещен на разных
вычислительных устройствах: GPU, TPU.

```
1 import tensor
2 import numpy as np
3
4 data = torch.tensor([[1, 2], [3, 4]])
5 # torch.tensor(np.arange(4).reshape(2,2))
6
7 data_zeros = torch.zeros((3, 3))
8 ones = torch.ones_like(data_zeros)
9 rand = torch.randn((2, 3))
10
11 print(rand.shape, rand.dtype)
```

ОПЕРАЦИИ

```
1 a = torch.tensor([1, 2])  
2 b = torch.tensor([3, 4])  
3 print(a + b) # *, /, -, +, **, %
```

```
> tensor([4, 6])
```


ИЗМЕНЕНИЕ РАЗМЕРНОСТИ

```
1 a = torch.randn(4, 4)
2 flat = x.view(16) #a.size()
3 batch = x.reshape(-1, 8)
```



ИЗМЕНЕНИЕ РАЗМЕРНОСТИ

```
1 a = torch.randn(4, 4)
2 b = a.T
3
4 print(a.view(4*4))
5 print(b.view(4*4)) # RuntimeError
6 print(b.reshape(4*4))
7 print(b.contiguous().view(4*4))
```

ИЗМЕНЕНИЕ РАЗМЕРНОСТИ

```
1 image = torch.zeros(3, 48, 48)
2 data = image.unsqueeze(0) # C, H, W
3 print(data.shape) # 1, C, H, W
```

```
> torch.Size([1, 3, 48, 48])
```



РАСПОЛОЖЕНИЕ В ПАМЯТИ

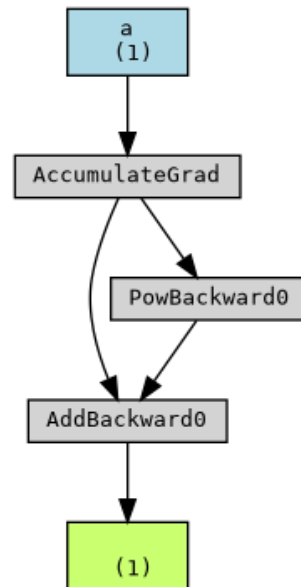
.to(device) - перенос тензора на устройство

```
1 print(torch.cuda.is_available())
2
3 a = torch.randn(4, 4)
4 a_gpu = a.to("cuda")
5
6 print(a_gpu)
7
8 print(a + a_gpu) # RuntimeError: Expected all tensors to be
9 print(a + a_gpu.cpu())
10
11 print(a_gpu + a_gpu)
```

АВТОМАТИЧЕСКОЕ ДИФФЕРЕНЦИРОВАНИЕ

`requires_grad=True` - отслеживание всех операций над тензором для последующего вычисления градиентов

```
1 a = torch.tensor([2.], requires_grad=True)
2
3 loss = a ** 2 + a
```



АВТОМАТИЧЕСКОЕ ДИФФЕРЕНЦИРОВАНИЕ

	requires_grad
Веса модели	True
Входные данные	False
Режим инференса	False
Заморозка слоев	False

Требует дополнительной памяти!



ВЫЧИСЛЕНИЕ ГРАДИЕНТОВ

```
1 a = torch.tensor([2.0], requires_grad=True)
2
3 x = a ** 2
4 x.backward()
5 print(a.grad)
6
7 y = a ** 3
8 y.backward()
9 print(a.grad)
```

```
tensor([4.])
tensor([16.])
```

ОТКЛЮЧЕНИЕ РАСЧЕТА ГРАДИЕНТОВ

```
1 a = torch.tensor([2.0], requires_grad=True)
2
3 with torch.no_grad():
4     x = a ** 2
5     y = a ** 3
6
7     print(a.grad)
```

```
1 None
```


ЗАЧЕМ НАКАПЛИВАТЬ?

- Работы с батчами

64 изображения - 4 батча по 16 - 4 раза
накопление градиента

- Несколько функций потерь

Расчет градиентов для Loss_1, расчет
градиентов для Loss_2, суммарная ошибка

НЕЙРОННАЯ СЕТЬ

Module - базовый класс для разработки архитектуры сети

```
1 import torch.nn as nn
2
3 class Model(nn.Module):
4     def __init__(self):
5         super().__init__()
6         self.flatten = nn.Flatten()
7         self.linear = nn.Linear(28*28, 20)
8
9     def forward(self, x):
10        x = self.flatten(x)
11        logits = self.linear(x)
12        return logits
13
14 model = Model()
15 print(model)
```



MODULE

- **forward** - прямой проход данных через модель
- **train** - режим обучения
- **eval** - режим оценки
- **to(device)** - перенос модели на устройство
- **zero_grad** - обнуление всех градиентов

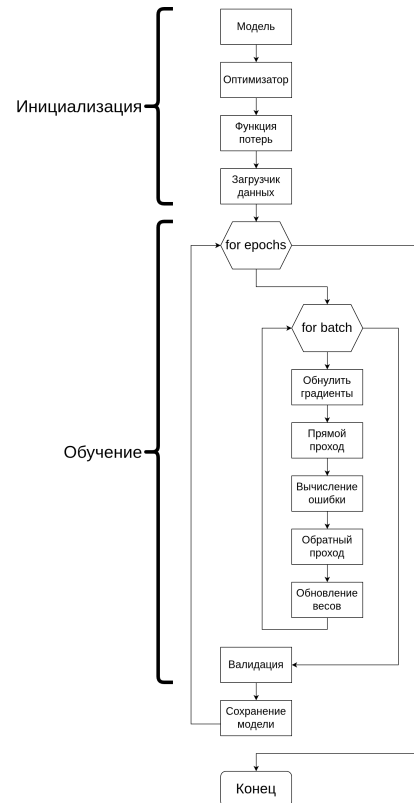
MODULE

```
1 model.state_dict()
```

```
OrderedDict([('linear.weight',  
            tensor([[ -0.0348,  0.0142,  0.0058, ...,  0.008  
                    [ 0.0049,  0.0056,  0.0285, ...,  0.023  
                    [ 0.0029,  0.0253, -0.0078, ...,  0.022  
                    ...,  
                    [ 0.0052,  0.0225, -0.0256, ..., -0.014  
                    [ 0.0200, -0.0213, -0.0038, ...,  0.021  
                    [ 0.0185, -0.0295,  0.0020, ..., -0.017  
('linear.bias',  
            tensor([ 0.0332, -0.0101,  0.0224, -0.0345, -0.0  
                    0.0214, -0.0166,  0.0195, -0.0164, -0.0  
                    0.0153,  0.0145, -0.0273, -0.0223]))]))
```



ЦИКЛ ОБУЧЕНИЯ



СЛОИ

- `nn.Linear`
- `nn.Conv2d`
- `nn.ReLU`, `nn.Sigmoid`, `nn.Softmax`
- `nn.Dropout`, `nn.BatchNorm`

LINEAR

Слой полносвязной нейронной сети

$$y = xW^T + b$$

- Агрегация признаков
- Изменение размерности

```
1 self.fc = nn.Linear(in_features=128,  
2                       out_features=10)
```

CONV2D

Свертка фильтра с изображением для вычисления взвешенной суммы пикселей

- Извлечение признаков
- Инвариантность к сдвигу
- Экономия параметров

```
1 self.conv = nn.Conv2d(in_channels=3,  
2                       out_channels=64,  
3                       kernel_size=3,  
4                       padding=1)
```


RELU

$$f(x) = \max(0, x)$$

Для чего	обнуляет отрицательные значения
----------	---------------------------------

Где	скрытые слоя
-----	--------------

Плюсы	быстрое вычисление
-------	--------------------

Минусы	мертвые нейроны
--------	-----------------



SIGMOID

$$f(x) = \frac{1}{1+e^{-x}}$$

Для чего сжимает в диапазон (0, 1)

Где классификация, сегментация

Минусы медленная, затухает на краях

SOFTMAX

$$f(x) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Для чего вектор чисел в вероятности

Где многоклассовая классификация

Torch Не использовать с CrossEntropyLoss



DROPOUT

Защита от переобучения путем обнуления нейронов с некоторой вероятностью

Для чего	улучшение обобщающей способности
----------	----------------------------------

Где	после Linear
-----	--------------

Torch	отключается во время инференса
-------	--------------------------------



BATCHNORM

Нормировка слоя активации по батчу

Для чего ускоряет сходимость

Где после Linear иди Conv2d

CV Conv - BatchNorm - ReLU

ФУНКЦИИ ПОТЕРЬ И ОПТИМИЗАТОРЫ

- `nn.CrossEntropy`, `nn.MSELoss`
- `optim.SGD`, `optim.Adam`



ФУНКЦИЯ ПОТЕРЬ

Что такое функция потерь?



ФУНКЦИЯ ПОТЕРЬ

Что такое функция потерь?

Расчет скалярного числа(ошибки) для определения того насколько предсказание модели отличается от истинного значение

CROSSENTROPYLOSS

Разница между распределением вероятностей
предсказания с истинным классом

$$H(p, q) = - \sum_x p(x) \log q(x)$$

Для чего	классификация, семантическая сегментация
-------------	---

Вход	предсказания и целевые классы
------	-------------------------------

MSELOSS

Средний квадрат разности между предсказанием и целью.

$$\frac{1}{N} \sum (y_{true} - y_{pred})^2$$

Для чего	регрессия, восстановление изображений
-------------	--

Вход	предсказания и целевые значения
------	---------------------------------

ОПТИМИЗАТОР

Что такое оптимизатор?



ОПТИМИЗАТОР

Что такое оптимизатор?

Алгоритм обновления весов модели на основе градиентов.



SGD

Градиентный спуск - движение весов в сторону, противоположную направлению градиента

$$w = w - lr \cdot \nabla L$$

Для чего	fine-tuning, максимальная точность
----------	------------------------------------

Плюсы	лучшая обобщающая способность, меньше памяти
-------	--

Минусы	чувствителен к learning rate, медленный
--------	---



ADAM

Для каждого параметра модели вычисляет свою скорость обучения.

Для чего	большинство задач, обучение с нуля, GAN
----------	---

Плюсы	быстрая сходимость
-------	--------------------

Минусы	требуется больше памяти
--------	-------------------------



ЗАДАНИЕ

Сделать модель для возведения числа в квадрат.
Без обучения и с обучением.



DATASET

Класс для создания ленивых наборов данных

```
1 from torch.utils.data import Dataset
2 import torch
3
4 class ImageDataset(Dataset):
5     def __init__(self, img_paths, labels):
6         self.img_paths = img_paths
7         self.labels = labels
8
9     def __len__(self):
10        return len(self.labels)
11
12    def __getitem__(self, idx):
13        img = torch.randn(3, 64, 64)
14        label = self.labels[idx]
15        return img, label
```


DATALOADER

Автоматизация загрузки данных, формирование батчей, перемешивания и многопоточности.

```
1 from torch.utils.data import DataLoader
2
3 loader = DataLoader(dataset, batch_size=2, shuffle=True)
4
5 for batch_idx, (data, target) in enumerate(loader):
6     print(batch_idx, data.shape, target)
```



ЗАДАНИЕ

Разработать Dataset который будет возвращать случайное изображение при запросе и его метку.

При помощи DataLoader проверить работоспособность класса.



ПОДГОТОВКА ДАННЫХ ДЛЯ ОБУЧЕНИЯ

Предобработка - фиксированные шаги по подготовке данных для обучения и предсказания

Аугментация - случайные изменения для расширения датасета при обучении



ПРЕДОБРАБОТКА

- ToImage - преобразование (H x W x C) к (C x H x W)
- Normalize(mean, std) - $\text{channel} = (\text{channel} - \text{mean}) / \text{std}$
- Resize(size) - изменение размера изображения
- CenterCrop(size) - вырезка центра



ГЕОМЕТРИЧЕСКИЕ АУГМЕНТАЦИИ

- `RandomHorizontal(Vertical)lFlip(p)` - случайный переворот
- `RandomRotation(angle)` - случайный поворот
- `RandomCrop(size)` - случайная область
- `RandomResizedCrop(size, scale)` - случайный масштаб
- `RandomAffine(degree, scale...)` - аффинное преобразование



ФОТОМЕТРИЧЕСКИЕ АУГМЕНТАЦИИ

- ColorJitter(brightness, contrast, saturation, hue) - случайное изменение яркости, контраста, насыщенности, оттенка
- Grayscale(n_channels) - перевод в оттенки серого
- GaussianBlur - размытие
- RandomInvert - инверсия цветов



ОБУЧЕНИЕ И ВАЛИДАЦИЯ

```
1 train_transform = transforms.Compose([
2     transforms.RandomResizedCrop(224),
3     transforms.RandomHorizontalFlip(),
4     transforms.ColorJitter(brightness=0.2, contrast=0.2),
5     transforms.ToImage(),
6 ])
7
8 val_transform = transforms.Compose([
9     transforms.Resize(256),
10    transforms.CenterCrop(224),
11    transforms.ToImage(),
12 ])
```

ЗАДАНИЕ

Разработать Dataset который будет выдавать заданное количество изображений с заданным набором аугментации.



